

Arquitetura e Especificação Técnica

1. Premissas, Limitações e Contexto

O desenvolvimento até o presente estágio restringiu-se estritamente à concepção e treinamento do motor de inteligência artificial (modelo preditivo). Devido à indisponibilidade de uma base de metadados reais referentes ao ambiente de produção durante a fase de modelagem, o algoritmo foi construído e validado utilizando um *dataset* público de transações financeiras (Kaggle).

Sendo o modelo entregue essencialmente desacoplado do sistema final, sua operacionalidade está condicionada à implementação de um *pipeline* de integração. Como consolidado por D. Sculley (2015), apenas uma pequena fração dos sistemas reais de Inteligência Artificial é composta pelo código do modelo em si; a infraestrutura ao redor necessária para suportá-lo é vasta e complexa. Sob essa premissa, este documento especifica a arquitetura de um ambiente de validação isolado, projetado com a finalidade exclusiva de construir essa infraestrutura inicial para homologar o fluxo de entrada e saída de dados, atestando o comportamento do modelo em um cenário operacional simulado.

2. Objetivo do Ambiente de Validação (POC)

O escopo primordial desta arquitetura é implementar e atestar a viabilidade técnica de um ecossistema capaz de suportar o modelo de Machine Learning. O ambiente deve garantir:

- **Inferência de Borda (Tempo Real):** Capacidade de interceptar requisições HTTP (payload JSON) e processar a classificação binária de risco sob demanda.
- **Persistência Transacional:** Registro imutável, seguro e estruturado de todas as transações injetadas e avaliadas para fins de auditoria do modelo.
- **Observabilidade Executiva:** Disponibilização de uma camada de telemetria e *Business Intelligence* (BI) para aferição do comportamento da IA em tempo de execução.

3. Topologia da Arquitetura de Validação

A solução foi arquitetada sob o paradigma de separação de responsabilidades, estabelecendo um fluxo síncrono para inferência e assíncrono para retenção analítica. A topologia distribui-se em quatro instâncias containerizadas:

- **Frontend (Simulador de Operação):** Interface de *mock* dedicada à injeção manual de dados de teste.
- **Backend (Motor de Inferência):** Serviço de exposição da API do modelo preditivo.
- **Database (Repositório de Logs):** Banco de dados relacional para armazenamento dos logs operacionais.
- **Dashboard (Painel de Monitoramento):** Plataforma de consumo gerencial das previsões geradas.

4. Defesa do Stack Tecnológico

A seleção do *stack* privilegiou aderência ao ecossistema Python nativo do modelo, isolamento de processos e prontidão arquitetural, assegurando que o ambiente de validação reflita as práticas de um ambiente de produção definitivo.

4.1. Backend: FastAPI

O FastAPI apresenta latência inferior ao Flask e arquitetura mais enxuta que o Django, consolidando-se como o padrão ouro da indústria para o envelopamento de APIs RESTful de *Machine Learning*.

4.2. Banco de Dados: PostgreSQL

A opção por implementar um banco de dados relacional corporativo já na fase de validação constitui uma decisão estratégica para anular atritos futuros.

4.3. Frontend (Simulador): Streamlit

Considerando o escopo estrito de validação da POC, alocar horas de engenharia na construção de um *frontend web* clássico configura um antipadrão de produtividade. O Streamlit atende com precisão ao requisito de fornecer uma interface ágil e reativa para o envio dos *payloads* de teste contra a API.

4.4. Monitoramento: Metabase

O componente de monitoramento obedece à rígida separação de funções da arquitetura: o Streamlit atua na injeção; o Metabase atua no consumo executivo. Com conexão nativa e leitura direta ao PostgreSQL, o Metabase viabiliza a

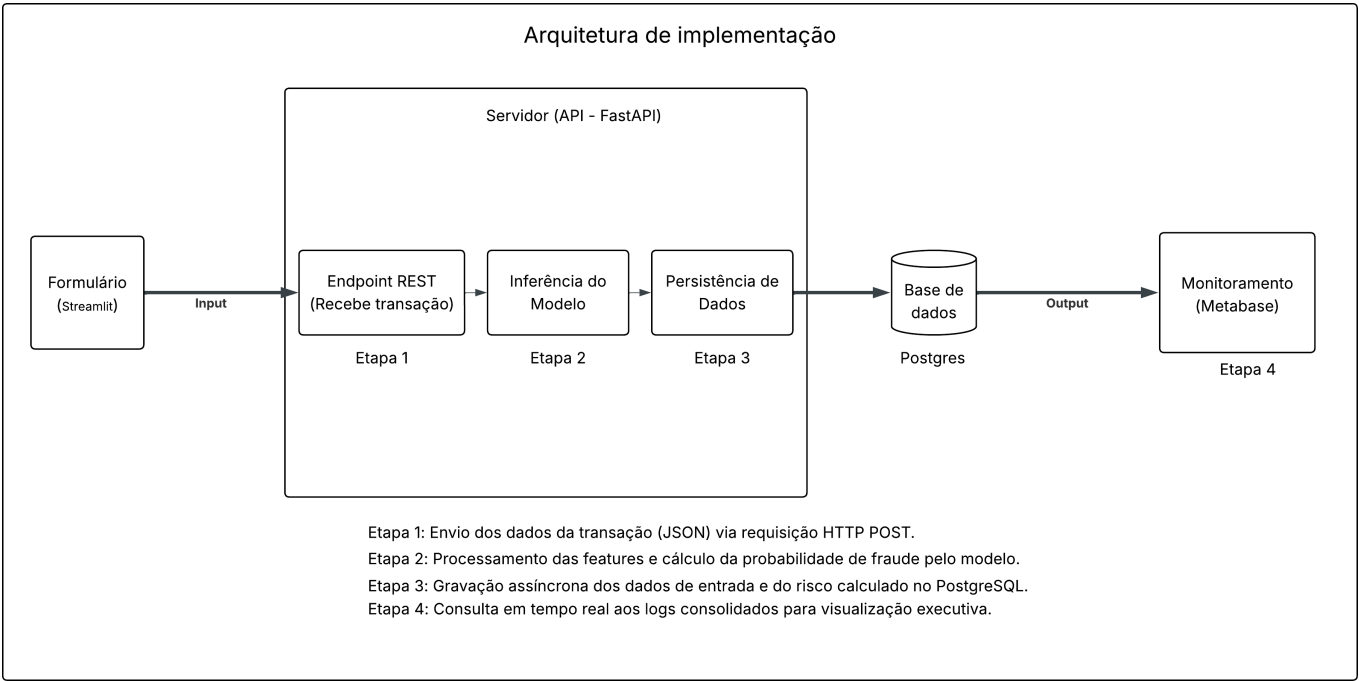
construção imediata de *dashboards* corporativos complexos (distribuição de score, anomalias detectadas) sem a escrita de código de visualização suplementar.

5. Fluxo de Dados de Homologação (Data Flow)

A execução de testes no ambiente de validação obedece ao seguinte pipeline:

1. O usuário submete um *payload* JSON contendo os metadados simulados da transação através da interface **Streamlit**.
2. A **FastAPI** intercepta a requisição HTTP, estrutura os tensores de entrada e aciona o modelo *Random Forest* (carregado em memória).
3. O modelo processa as variáveis e devolve o *score* avaliado (0.0 a 1.0 de probabilidade de anomalia).
4. O **FastAPI** persiste o registro integral da operação (Features de Entrada + Score + Timestamp) no **PostgreSQL**.
5. De forma autônoma, o **Metabase** consulta periodicamente a base de dados do PostgreSQL, atualizando os indicadores de risco e comportamento para visualização da diretoria.

Diagrama de arquitetura:



6. Estrutura e Topologia do Repositório

Para garantir a modularidade exigida por uma arquitetura de modular, o código-fonte está estruturado em contêineres lógicos e isolados. Esta organização facilita o

provisionamento automatizado (via Docker) e delimita claramente as responsabilidades de engenharia de software e ciência de dados.

```
/projeto_dec_anomalias
|
|— .env                # Variáveis de ambiente
|— .gitignore          # Exclusão
|— docker-compose.yml  # Orquestrador - Docker
|— README.md
|
|— /api                # Servidor (FastAPI)
|   |— Dockerfile
|   |— requirements.txt # Dependências estritas do backend
|   |— main.py          # Definição de rotas
|   |— database.py      # Camada de abstração e persistência
|   |— schemas.py       # Contratos de dados e validação
Pydantic
|   |— /modelos         # Artefatos estáticos de Machine
Learning
|       |— modelo_rf.pkl # Objeto serializado do modelo de ML
|
|— /frontend           # Interface de Simulação e Validação
    |— Dockerfile      # Docker
    |— requirements.txt # Dependências do frontend
    |— app.py           # Lógica de interface e cliente HTTP
```

7. Escopo e limitações

O desenvolvimento focou estritamente na criação da inteligência artificial (o modelo de inferência). Como não houve acesso aos dados reais do PDAF, a prova de conceito foi viabilizada utilizando um *dataset* aberto do Kaggle para mapear o comportamento transacional.

O modelo entregue é desacoplado do sistema final, ou seja, ele precisa de um fluxo externo de entrada e saída de dados para funcionar. Dessa forma, a implementação operacional em produção fica condicionada ao desenvolvimento da integração pelo sistema que irá requerer essas previsões.

8. Referências Técnicas e Bibliografia de Apoio

- HUYEN, Chip. *Designing Machine Learning Systems*. O'Reilly Media, 2022.
- BURKOV, Andriy. *Machine Learning Engineering*. True Positive Inc., 2020.
- KLEPPMANN, Martin. *Designing Data-Intensive Applications*. O'Reilly Media, 2017.
- NEWMAN, Sam. *Building Microservices: Designing Fine-Grained Systems*. O'Reilly Media, 2021.
- TREUIL, A., THIEBAUT, A., & AMMAR, T. *Machine Learning Applications Using Streamlit*. Apress, 2022.